

תרגיל בית 3 - מבוא למחשוב ענן

צוות - Zebra



GreenZebra

פרח שנבחר: סחלב



 GreenZebra	קישור למחברת הגשה:
GitHub	קישור לתיקיית גיט:

שם חבר הצוות	משימות שהוקצו	משימות שהושלמו	Acceptance test – בדיקת קבלה
טדי חדאד	מהנדס מערכת מימוש צ'אטבוט חלק שני סעיף 5,6	הושלם	צ'אטבוט עונה על שאלות בסיס לגבי מערכת
דניל גסן	RAG LLM + תיק מתכנת ותיק משתמש	הושלם	הרצת שאילתות למנוע חיפוש, תוצאה מתקבלת ועונה על שאלת המשתמש
לאון סיגל	פונקציונאליות חיפוש בדאטהסט	הושלם	מערכת מבצעת זיהוי לפי תמונה של סוג סחלב
ירין חממי	חלק שני סעיפים 4	הושלם	מערכת מציגה גרף עם חיישנים לפי זמן לאורך 3 ימים
תומר לוי	חלק שני סעיפים 2,3	הושלם	אין בדיקת קבלה

1. מימוש מלא של מנוע חיפוש:

בוצע שינוי במנוע חיפוש שכולל כרגע גם את TF-IDF, שנותנת לכל מאמר ציון על סמך כמה הוא מתאים לשאלת המשתמש. בנוסף בוצע חיבור של LLM חיצוני, בחרנו במודל GEMINI כאשר המודל שבחרנו זה Gemini 2.5 Flash, מודל שהוא חינוני ונותן תשובות שעונות לצרכי משתמש בשלב זה. כאשר ה LLM מקבל בשאלתה את הפרטים של חיישנים ביחד עם שאלת המשתמש למתן תשובה מיטבית, זאת השאלתה שמקבל AI:

```
prompt = (  
    'You are an expert orchid care assistant.\n'  
    'Answer the user question using the research excerpts  
below as your primary knowledge source.\n'  
    'Factor in the live IoT sensor readings to tailor your  
advice to the plant\'s actual current conditions.\n'  
    'Be concise, practical, and use bullet points where  
helpful.\n'  
    'Cite article numbers when referencing specific  
findings.\n\n'  
    f'User question: {query}\n\n'  
    f'Current sensor readings:\n{iot_block}\n\n'  
    f'Research excerpts:\n{article_context}'  
)
```

ככה נראית דוגמה של מתן תשובה הרצת שאלתה בסיסית:

The screenshot displays the GreenZebra web application. At the top, there's a navigation bar with tabs: Dashboard, Orchid Type Scanner, Live Sensors, and Research (which is highlighted). Below the navigation bar, the main content area is titled 'Research Query' with a subtitle 'Search across 5 indexed orchid research articles using our RAG system'. A search input field contains the text 'My orchid is looking dry', and a 'Search -->' button is next to it. Below the search bar, the 'AI ANSWER' section provides a response: 'Your orchid is likely looking dry due to the high temperature, despite adequate soil moisture. Here's what to do:'. It lists three bullet points: 'Address High Temperature' (current 37.2°C is too high, ideal is ≤30°C), 'Lower Temperature' (move to a cooler location), and 'Watering & Humidity' (current soil moisture 57.0% and air humidity 42.0% are within acceptable ranges). At the bottom, the 'LIVE SENSOR CONTEXT' section shows two sensor readings: 'Temperature: 37.2°C — TOO HIGH (ideal ≤30°C)' with a red warning icon, and 'Air Humidity: 42.0% — OK' with a green checkmark icon.

חיבור Hugging Face:

בנוסף בוצע חיבור לדאטהסט הקיים של סחלבים עם HuggingFace, כך נראת דוגמה של העלאת תמונה של סחלב ומערכת נותנת זיהוי על סמך סוג סחלב:



image_2026-06-28_142036932.png

Scan orchid →

Remove

ANALYSIS RESULT

🌸 Type: C. goeringii

2. שימוש ב-Microservices:

שירות אנליטיקה וניטור:

- במה עשינו שימוש: שירות זה מקבל כקלט נתונים מחיישן האדמה, מבצע עיבוד מתמטי לחישוב ממוצע הלחות, ומחזיר את הסטטוס הבריאותי המעודכן של העציץ.
- היתרון בשימוש בשירות: ניתוק צימוד בין לוגיקת העיבוד והניתוח לבין ממשק המשתמש ותהליך שליפת הנתונים מהמסד. היתרון המרכזי הוא מודולריות, ניתן בעתיד להרחיב את אלגוריתם הניתוח אך ורק בתוך המיקרו שירות, מבלי להשפיע או לסכן את יציבות הקוד של שאר חלקי המערכת.

שירות משחוק:

- במה עשינו שימוש: שירות מבודד המנהל את מערכת הניקוד. הוא מקבל מהמערכת דיווח על סוג הפעולה שבוצעה (למשל השקיה או בדיקת חיישנים), מחשב את הניקוד המתאים לפי החוקיות המוגדרת בו, ומחזיר את הערך המעודכן לצורך שמירתו ב-DB.
- היתרון בשימוש בשירות: בידוד ה Business Logic של מנגנון המשחוק. אם נרצה לבצע שינויים או הוספות למשחוק, נעשה את זה רק בתוך המיקרו שירות. זה בעצם מאפשר מינוף עתידי נוח במידה ומערכת המשחוק תגדל ותצרוך יותר משאבי עיבוד.

3. KPI רלוונטיים בפרויקט:

ציון בריאות הצמח: מדד המחושב באחוזים מ-0% עד 100% ומשקף את תקינות סביבת הגידול בזמן אמת. הציון מחושב על ידי יחס החיישנים של הטמפרטורה, לחות, ואדמה, שקריאותיהם נמצאות בתוך טווחים אידיאליים המוגדרים מראש. המדד מתורגם לאינדיקציה חזותית המראה אם המצב מצוין, טוב, דורש תשומת לב או קריטי.

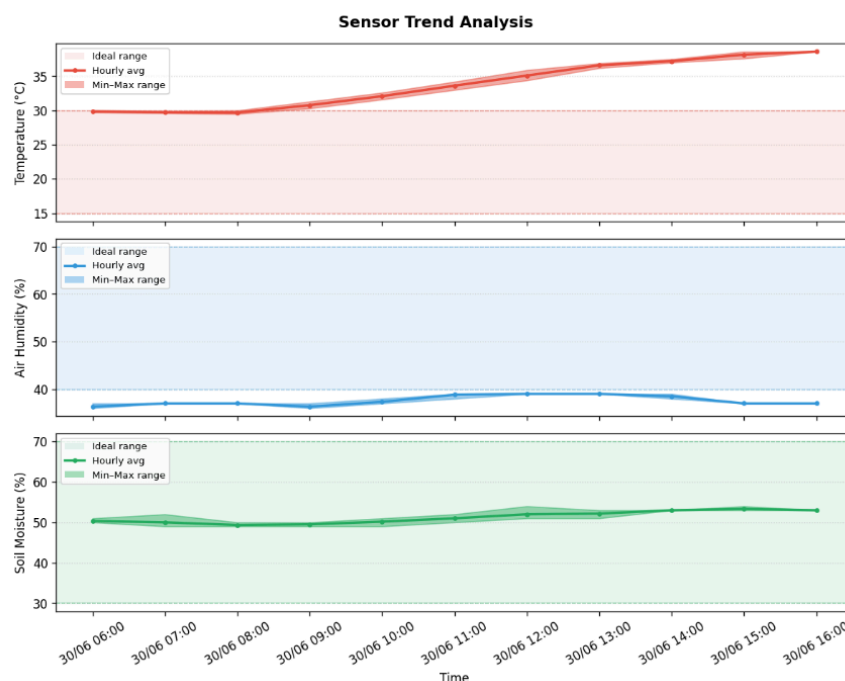
רצף בטיפול: מדד על המשתמש העוקב אחר מספר הימים הרצופים בהם בוצעה בדיקה או השקיה. המדד הוא חלק ממערכת המשחוק המאפשרת מעקב התקדמות וקבלת badges. זהו KPI חשוב לבדיקת שימור משתמשים ועקביות בטיפול הסחלב.

Uptime & Latency:

- **Uptime:** זהו המדד שבודק כמה זמן השרת שממנו אנחנו מביאים את נתוני החיישנים באמת עובד ונמצא באוויר. בקוד יש מנגנון הגנה (Timeout) שממתין לנתונים, מכיוון שהשרת החיצוני החינמי לפעמים "הולך לישון" כשלא משתמשים בו ולוקח לו זמן להתעורר. מדד זה קריטי כי אם השרת לא זמין, המערכת לא יכולה לקרוא נתונים מהחיישנים.
- **Latency:** המדד שבודק כמה זמן לוקח למידע להגיע מהרגע שהמערכת ביקשה אותו ועד שהוא מוצג למשתמש. אנחנו רוצים שהתגובה תהיה מהירה ככל הניתן כדי שהמשתמש לא יחכה למסך טעינה ארוך, ושציון בריאות הצמח והמלצות ה-AI יתעדכנו מיד.

4. גרף של נתוני החיישנים לאורך זמן

ציר הזמן מתעדכן באופן דינמי בהתאם לטווח הנתונים הקיים ב-DB, הגדרנו מקסימום 12 תוויות שיוצגו בציר הזמן כדי לשמור על קריאות הגרף ולמנוע צפיפות יתר של התוויות.



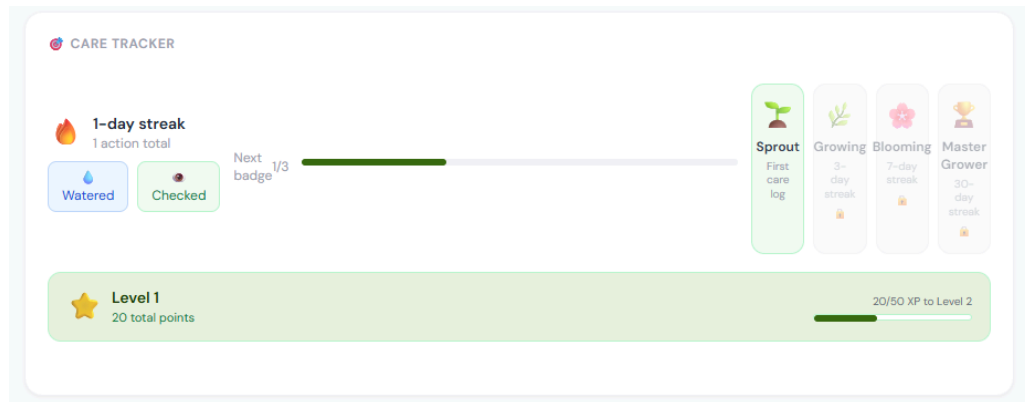
5 - 6. אתגרים בפרויקט וכיצד התמודדנו איתם כצוות

אחד האתגרים המרכזיים שלנו בפרויקט הוא הכמות הגדולה של הדאטה, כלומר בניית האינדקס לחיפוש המאמרים (TF-IDF) דרשה מאיתנו לחשוב כיצד לארגן את הנתונים ב-Firebase בצורה שתאפשר שליפה מהירה ויעילה, בלי להעמיס יותר מדי על כל שאילתה, כאשר אפילו נתקלנו במצב שבו סיימנו את המגבלה היומית ב-Firebase לאחר הרצות חוזרות ועדכונים קטנים של הקוד. בנוסף, שילוב הוויזואליזציה בתוך הממשק היה מאתגר במיוחד בסביבת Colab, מכיוון שכל תא רץ מבודד משלו, כך שלא יכולנו פשוט להציג רכיב שנבנה בתא נפרד בתוך האפליקציה הראשית. נאלצנו לאחד את כל הלוגיקה לתוך תא אחד מרכזי שמכיל גם את ה-HTML/JS וגם את ה-Python, כדי שהכל ירוץ באותה סביבה ויוכל לתקשר זה עם זה.

אתגר נוסף היה מבנה הקוד עצמו – מכיוון שכל הממשק בנוי כמחרוזת HTML/JS ענקית בתוך Python, כל שינוי קטן דרש זהירות רבה כדי לא לשבור את הסנכרון בין האלמנטים בדף לבין הפונקציות שמטפלות בהם בצד הפייתון. כצוות, התמודדנו עם זה על ידי שמירה על מבנה מסודר ועקבי של שמות ו-ID-ים ובדיקות הדרגתיות אחרי כל שינוי כדי לוודא שהפונקציונליות הקיימת לא נפגעת.

אלמנט המשחק:

לבסוף, אלמנט המשחק שהוספנו למערכת כולל מערכת נקודות ורמות המבוססת על פעולות הטיפול בצמחים ועל השימוש בצ'אטבוט, כאשר כל פעולה (השקיה, בדיקה, או שאלה לעוזר ה-AI) מזכה בנקודות שמצטברות לרמות עם פס התקדמות (XP), לצד מערכת התגים הקיימת המבוססת על רצף ימים רצוף של טיפול בצמח.



7. בונוס

ממשנו לוגיקת סוכן גם בצ'אטבוט וגם בחלק של RAG, הסוכן בחלק של ניתוח מידע על פרח מקבל את כל התונים על פרח ויודע לתת הצעות לשיפור הכי טובות. בנוסף הצ'אטבוט מוסיף נקודות למשתמש אם הוא מדבר איתו על הפרחים שלו, שזה מגביר ידע אצל משתמש על פרחיו

חלק שלישי: תיק מתכנת

1. פונקציות מרכזיות:

מנוע הטוקניזציה — `tokenize(text)`

מטרה: ממירה טקסט גולמי לרשימת טוקנים מנורמלים המוכנים לאינדוקס.

תהליך הפעולה:

1. המרת הטקסט לאותיות קטנות.
2. החלפת תווי פיסוק (נקודות, פסיקים, סוגריים) ברווחים.
3. סינון מילות עצור (Stop Words) ומילים קצרות מ-3 תווים.
4. למאטיזציה (הוצאת שורש דקדוקי של כל מילה).

בניית אינדקס TF-IDF — לוגיקת האינדוקס (תאים 3-4)

מטרה: בניית אינדקס מהיר לחיפוש מאמרי מחקר לפי רלוונטיות.

תהליך הפעולה:

1. טוקניזציה של 5 מאמרי מחקר.
2. חישוב תדירות מסמכים (DF).
3. חישוב IDF מוחלק למניעת אפס-חלוקה.
4. חישוב TF-IDF לכל זוג (מונח, מסמך).

5. אחסון האינדקס ב-Firebase Firestore.

שליפת נתוני חיישנים `fetch_sensor_data(feed, limit)`

מטרה: שולפת קריאות היסטוריות ממשרת ה-IoT על הענן.

פרמטרים:

`feed` סוג החיישן: "temperature", "humidity", "soil", "json" (כל החיישנים).

`limit` מספר הדגימות לשליפה (1–50).

טיפול בשגיאות: מחזירה ([], הודעת שגיאה) בשני מקרים: (timeout) ושגיאה כללית, מבנה זה מאפשר למשתמש לראות הודעה מובנת ולא קריסה.

`_build_rag(query)` — RAG (Retrieval-Augmented Generation) מנוע

מטרה: ממשק חיפוש חכם המשלב אינדקס TF-IDF עם מודל של Gemini.

תהליך בן 5 שלבים:

1. טוקניזציה ולמאטיזציה של שאילתת משתמש.
2. שליפת ציוני TF-IDF מ-FIREBASE לכל מונח בשאילתה
3. דירוג מסמכים לפי ציון מצטבר, בחירת 2 מסמכים מובילים
4. שליפת קריאות חיישנים
5. בניית פרומפט עבור מודל GEMINI וקבלת מענה

`_build_overview_and_advice()` — ציון הבריאות של הסחלב

מטרה: מחשבת ציון בריאות (0–100%) על בסיס 3 חיישנים.

לוגיקת הציון:

1. בודקת כל חיישן מול טווחי הסף המוגדרים ב-THRESHOLDS.
2. כל חיישן בטווח $1+ =$ לנקודת הבריאות.
3. הציון הסופי $= (\text{מספר חיישנים תקינים} / 3) \times 100$.

`_build_gamification()` — מערכת גמיפיקציה

מטרה: מדידת רצף טיפול יומי ומתן תגים.

לוגיקת חישוב הרצף:

1. שולפת את כל רשומות care_log מ-Firestore.
2. ממיינת תאריכים בסדר יורד.
3. בודקת רציפות: האם כל יום קיימת רשומה, החל מהיום.
4. אם יום חסר, הרצף נשבר.

תבניות עיצוב בפרויקט:

TF-IDF עם IDF מוחלק

```
idf_scores = {term: math.log((N + 1) / (cnt + 1)) + 1 for term, cnt in
df.items() }
```

```
# Step 4: build TF-IDF index
```

```
tfidf_index = defaultdict(list)
```

```
for doc_id, data in doc_tokens.items():
```

```
    for term, count in data['token_counts'].items():
```

```
        tf = count / data['total_tokens']
```

```
        score = round(tf * idf_scores[term], 8)
```

```
        if score > 0:
```



```
tfidf_index[term].append({'doc_id': doc_id, 'url': data['url'],
'tfidf': score})
```

הנוסחה הסטנדרטית שוברת כאשר מונח מופיע בכל המסמכים. הגרסה המוחלקת מוסיפה 1+ למונה ולמכנה וכן 1+ לתוצאה, מה שמבטיח שאף מונח לא יקבל ציון אפס.

המרת HTML-ל-Markdown

```
import re as _re

text = text.replace('&', '&amp;').replace('<', '&lt;').replace('>', '&gt;')

text = _re.sub(r'\*\*(.*?)\*\*', r'<strong class="font-semibold
text-gray-800">\1</strong>', text)

lines = text.split('\n')

parts = []

in_list = False

for line in lines:

    s = line.strip()

    is_bullet = s.startswith('* ') or s.startswith('- ') or s.startswith('• ')

    if is_bullet:

        if not in_list:

            parts.append('<ul class="flex flex-col gap-1.5 my-1">')

            in_list = True

        content = s[2:]

        parts.append(

            '<li class="flex items-start gap-2 text-sm text-gray-700">'

            '<span class="text-green-500 mt-0.5 shrink-0 font-bold">•</span>'

            f'<span>{content}</span></li>'

        )

    else:

        if in_list:

            parts.append('</ul>')
```

```
in_list = False

if s == '':

    parts.append('<div class="h-1.5"></div>')

else:

    parts.append(f'<p class="text-sm text-gray-700 leading-relaxed">{s}</p>')

if in_list:

    parts.append('</ul>')

return '\n'.join(parts)
```

במקום להשתמש בספריית Markdown חיצונית, כתבנו ממיר מינימלי בהתאמה אישית.

תיק משתמש

הסבר כללי על המערכת

Green Zebra היא מערכת חכמה לניטור וטיפול סחלבים (Orchids), הפועלת כאפליקציית ווב בתוך Google Colab. המערכת מחברת ארבעה עולמות טכנולוגיים:

רכיב	מה הוא עושה בשבילך
חיישני IoT	מודדים בזמן אמת את הטמפרטורה, לחות האוויר ולחות הקרקע סביב הסחלב שלך
בינה מלאכותית (Gemini)	עונה על שאלות שלך לגבי טיפול בסחלב, בהתאם למצב הסחלב כרגע
זיהוי תמונות	מזהה את סוג הסחלב שלך מתמונה שתעלה
מחקר מדעי (TF-IDF + RAG)	מחפש בין 5 מאמרי מחקר אקדמיים תשובות לשאלותיך

פירוט מסכים (Screens Breakdown)

Dashboard

זהו המסך הראשי שנטען בפתיחת האפליקציה.

מה המשתמש רואה:

- 1. ציון בריאות (Health Score) בסקלה 0-100%, ולצדו קריאות חיות של שלושת החיישנים.
- 2. המלצות טיפול אוטומטיות מבוססות על מצב החיישנים הנוכחי.
- 3. מעקב רצף טיפול יומי עם תגים לפי הישגים.

פעולות אפשריות:

פעולה	מה קורה
לחיצה על "Refresh sensors"	שולף נתוני חיישנים עדכניים ומחשב מחדש את ציון הבריאות
לחיצה על "Watered"	מתעד שהצמח הושקה היום ב-Firestore
לחיצה על "Checked"	מתעד בדיקה יומית של הצמח

ציוני הבריאות:

ציון	סמל	משמעות
100%		Excellent
67-99%		Good
34-66%		Needs attention
0-33%		Critical

תגי ההישגים:

תג	תנאי
 Sprout	רישום טיפול ראשון

רצף 3 ימים	 Growing
רצף 7 ימים	 Blooming
רצף 30 ימים	 Master Grower

Orchid Type Scanner

מה המשתמש רואה:

1. אזור "גרור ושחרר" להעלאת תמונה.
2. תמונה ממוזערת (Thumbnail) לאחר בחירת תמונה.
3. תוצאת הזיהוי יהיה שם מדעי של הסחלב.

פעולות אפשריות:

פעולה	אופן ביצוע
העלאת תמונה	גרור לאזור הייעודי או לחץ "browse"
ביצוע סריקה	לחץ "Scan orchid"
הסרת תמונה	לחץ "X" על התמונה או "Remove"

Live Sensors

מה המשתמש רואה:

1. כפתורי בחירת חיישן (טמפרטורה, לחות, לחות קרקע, כל החיישנים).
2. סליידר לבחירת מספר הדגימות (1–50).
3. טבלת קריאות עם סטטוס לכל קריאה.
4. ממוצע, מינימום, מקסימום.
5. המלצות התראה אם ערכים חורגים מהטווח האידיאלי.
6. גרף מגמות שעות.

Research

מה המשתמש רואה:

1. שדה חיפוש חופשי.
2. תשובה של בינה מלאכותית מבוססת מאמרים.
3. נתוני חיישנים חיים כהקשר.
4. רשימת מקורות מדורגת לפי רלוונטיות TF-IDF עם קישורים למאמרים המקוריים.

מעברים בין מסכים

מנגנון הניווט:

הניווט מתבצע דרך שורת לשוניות (Tabs) בראש הדף.
המעבר מיידי (ללא טעינת עמוד מחדש) — JavaScript מסתיר/מציג <div> בהתאם.
בלחיצה ראשונה על Dashboard, נטען אוטומטית פרופיל ה-Care Tracker מ-Firebase.

זרימת המשתמש המומלצת:

1. פתיחה Dashboard : ראה ציון בריאות כללי
2. לחץ "Refresh": כדי לקבל עצות AI
3. עבור ל-Live Sensors: כדי לבדוק נתונים מפורטים
4. עבור ל-Scanner : כדי לזהות את סוג הסחלב
5. עבור ל-Research: כדי לשאול שאלות מקצועיות

טעויות אפשריות

שגיאה 1: פסק זמן של השרת (Server Timeout)

הודעה: "Server timeout – it may be waking up, please try again".
סיבה: שרת החיישנים מתארח בשירות Render.com בדרגה חנימית, שמכניס אותו ל"שינה" לאחר חוסר פעילות. ההתעוררות עשויה לקחת עד 30 שניות.

פתרון למשתמש: המתן 30 שניות ולחץ שוב על "Fetch readings".

שגיאה 2: קובץ לא חוקי (Invalid File Type)

שגיאה: "(Only image files are allowed (PNG, JPG, WEBP, etc)".
סיבה: הועלה קובץ שאינו תמונה (PDF, Word וכד').
פתרון למשתמש: העלה תמונה בפורמט PNG, JPG, JPEG, WEBP או GIF.

שגיאה 3: תמונה גדולה מדי

הודעה: "Maximum file size is 10 MB".

סיבה: גודל הקובץ חורג מ-10 מגה-בייט.

פתרון למשתמש: הקטן את התמונה לפני ההעלאה.

שגיאה 4: העלאת יותר מתמונה אחת

הודעה: "Only one image can be uploaded at a time".

סיבה: המשתמש ניסה לגרור מספר קבצים בו-זמנית.

פתרון למשתמש: העלה תמונה אחת בלבד.

שגיאה 5: שאליתת חיפוש ריקה או קצרה מדי

הודעה: "Query too short. Try: orchid, humidity, disease, growth ⚠️"

סיבה: כל מילות השאלית הוסרו בסינון (מילות עצור או מילים קצרות מ-3 תווים).

פתרון למשתמש: השתמש במונחים מדעיים ספציפיים יותר.

שגיאה 6: לא נמצאו מאמרים תואמים

הודעה: "No matching articles found. Try: orchid, humidity, disease, growth ⚠️"

סיבה: המונחים שבשאלית אינם מופיעים באף אחד מ-5 מאמרי המחקר שאינדקסו.

פתרון למשתמש: נסח את השאלית במונחים בוטניים או חקלאיים באנגלית.

שקיפות אלגוריתמית

נתונים הנאספים

1. נתוני חיישני IoT

מה נאסף: כל לחיצה על "Fetch readings" שולפת קריאות מחיישנים פיזיים ושומרת אותן ב-Firebase Firestore בקולקציה `sensor_readings`.

2. תמונות הסחלב

מה נאסף: התמונה שהמשתמש מעלה ממירה ל-Base64 ומועברת ישירות ל-Python בתוך הסשן. יש פירוט מסודר למשתמש איך עדיף לצלם תמונה ומה הוא יקבל בתוצאה

3. יומן הטיפול (Care Log)

כל לחיצה על "Watered" או "Checked" שומרת רשומה ב-Firebase Firestore:

יש הסברים למשתמש איך הוא יכול להגדיל את הדירוג שלו